Day 5 of 15 · Core Unit: Prepare Data Sets · LO2

Extract and Combine Data: Aggregates, JOINs and Export to Excel

Student Handout · TESDA Data Analytics Level III · Batch DAL3-2026

Case company: **Tindahan ni Aling Nena Mini-Mart** · combine sales with products and branches, then hand the data to Excel

Learning Outcomes

By the end of Day 5 you should be able to:

1. Summarize a whole table with `COUNT`, `SUM`, `AVG`, `MIN`, `MAX` and `ROUND`.
2. Give columns and tables short names with **aliases** (`AS`).
3. Combine two or three tables with `INNER JOIN … ON`.
4. Find "missing matches" (products never sold, branches without sales) with `LEFT JOIN` and `IS NULL`.
5. **Export** a query result to **CSV** and **import** it into **Excel** with *Data → From Text/CSV*.
6. Check that the totals in Excel **match** the totals in SQL.
7. Work as a **driver/navigator pair** and keep a pair log. *(Integrated: Lead small teams.)*

Materials you need today

Item	File / Location
SQL lab folder (open this folder in VS Code)	`Day05/assets/Day05_SQL_Lab/`
Your query script	`Day05_SQL_Lab/Day05_Queries.sql`
Demo video	`Day05/demo/Day05_Demo_Joins_and_Export_to_Excel.mp4`
Activity sheets + pair log	`Day05_Student_Activities.pdf`

> **Yesterday's tools, today's power.** Open the folder exactly like Day 4 (**Ctrl + K, Ctrl + O**), select a query, run it with **Ctrl + E, Ctrl + E**.

Part 1 — Aggregate Functions: One Answer from Many Rows

> **Sa simpleng salita:** An **aggregate** squeezes many rows into **one number**, like counting all the receipts in a box, or adding up all their amounts. 500 sales go in, one answer comes out.

Function	What it gives	Example	Result
`COUNT(*)`	number of rows	`SELECT COUNT(*) FROM sales;`	500
`COUNT(column)`	number of rows where the column is **not NULL**	`SELECT COUNT(customer_id) FROM sales;`	192

Function	What it gives	Example	Result
`COUNT(DISTINCT column)`	number of **different** values	`SELECT COUNT(DISTINCT customer_id) FROM sales;`	40
`SUM(column)`	total	`SELECT SUM(total_amount) FROM sales;`	49319.0
`AVG(column)`	average (mean)	`SELECT AVG(total_amount) FROM sales;`	98.638
`MIN` / `MAX`	smallest / biggest	`SELECT MIN(unit_price), MAX(unit_price) FROM products;`	12.0, 245.0
`ROUND(x, 2)`	round to 2 decimals	`SELECT ROUND(AVG(total_amount), 2) FROM sales;`	98.64

With WHERE: the filter runs **first**, then the aggregate.

```
SELECT SUM(total_amount) FROM sales WHERE payment_method = 'GCash';    -- 14253.0
```

> **Same as Excel!** `COUNT(*)` ≈ `=COUNTA()`, `SUM` ≈ `=SUM()`, `AVG` ≈ `=AVERAGE()`, `MIN/MAX` ≈ `=MIN()/=MAX()`. And `WHERE` + `SUM` ≈ `=SUMIF()`. On Day 9 we will add `GROUP BY` to get one total per branch in a single query.

Part 2 — Aliases: Short, Clear Names

> **Sa simpleng salita:** An **alias** is a **nickname**. `AS total_revenue` gives the answer column a readable heading. `sales AS s` lets you write `s` instead of `sales`, like calling *Jose Protacio* "Pepe".

```
SELECT SUM(total_amount) AS total_revenue FROM sales;        -- column alias: the heading becomes total_revenue
SELECT s.sale_id, s.total_amount FROM sales AS s;              -- table alias: s means sales
```

- Use **lower_case_with_underscores** for column aliases (no spaces), so Excel and Power BI read them easily.
- `AS` is optional (`sales s` also works), but writing it is clearer for beginners.

Part 3 — INNER JOIN: Combine Tables That Match

> **Sa simpleng salita:** The `sales` table only says "*product 107*". The name *Argentina Corned Beef 150g* is in the `products` table. A **JOIN** puts the two tables **side by side** and matches each sale to its product using the shared ID, like matching each receipt to the price list using the item code.

```
SELECT s.sale_id, s.sale_date, p.product_name, s.quantity, s.total_amount
FROM sales AS s
INNER JOIN products AS p ON s.product_id = p.product_id;
```

sale_id	sale_date	product_name	quantity	total_amount
TXN-10001	2026-01-05	Argentina Corned Beef 150g	2	84.0
TXN-10002	2026-01-05	Nissin Cup Noodles	2	56.0
TXN-10003	2026-01-05	Load - Globe/Smart P100	1	100.0

How to read it: *"From sales (call it s), join products (call it p), where the sale's product_id equals the product's product_id."*

Part	Meaning
`FROM sales AS s`	start from the sales table
`INNER JOIN products AS p`	bring in the products table
`ON s.product_id = p.product_id`	the **matching rule**: foreign key = primary key
`p.product_name`	"the product_name column of p". The prefix tells SQL which table, because both tables can have a column with the same name

INNER JOIN keeps only rows that match on both sides. All 500 sales have a product, so you get 500 rows.

Three tables: the joined dataset

```
SELECT s.sale_id, s.sale_date, b.branch_name, p.product_name, p.category,
       s.quantity, s.unit_price, s.total_amount, s.payment_method
FROM sales AS s
INNER JOIN branches AS b ON s.branch_id = b.branch_id
INNER JOIN products AS p ON s.product_id = p.product_id
ORDER BY s.sale_id;
```

One `JOIN … ON …` per extra table. This gives a **flat, readable table** (500 rows, 9 columns), the kind Excel and Power BI love.

Join + WHERE + aggregate answers business questions:

```
-- total revenue of the Beverages category
SELECT SUM(s.total_amount) AS beverages_revenue
FROM sales AS s
INNER JOIN products AS p ON s.product_id = p.product_id
WHERE p.category = 'Beverages';                          -- 3244.0
```

Part 4 — LEFT JOIN: Find What Is Missing

> **Sa simpleng salita:** A **LEFT JOIN** keeps **every row of the left table**, even if it has no partner on the right. Where there is no partner, the right side is **NULL** (empty). It's like taking attendance: every student on the class list is shown, and the ones without a "present" mark are the absentees.

```sql
-- products that have never been sold
SELECT p.product_id, p.product_name
FROM products AS p                          -- LEFT table: keep ALL products
LEFT JOIN sales AS s ON p.product_id = s.product_id
WHERE s.sale_id IS NULL;                    -- no matching sale = never sold
```

product_id	product_name
131	Kopiko Blanca Twin Pack
132	Spam Classic 340g

	INNER JOIN	LEFT JOIN
Keeps	only rows that match on **both** sides	**all** rows of the left table
No match	row is dropped	right-side columns are NULL
`branches` ⋈ `sales`	500 rows (Antipolo disappears)	501 rows (Antipolo appears once, with NULL sale)
Typical question	"Show sales with their product names"	"Which products/branches/customers have **no** sales?"

> **The left table is the one in FROM.** `FROM products LEFT JOIN sales` keeps all products. `FROM sales LEFT JOIN products` keeps all sales, which is a different question.

Part 5 — Export to CSV and Import into Excel

> **Sa simpleng salita: CSV** (comma-separated values) is a plain text file where each row is one line and the values are separated by commas. Almost every program can read it, so it is the "common language" for moving data between SQL, Excel and Power BI.

Export from VS Code (SQLTools)

1. Run the joined-dataset query (Q11).
2. At the bottom of the result panel, click **EXPORT** → **Save results as CSV**.
3. The *Save As* window opens in your lab folder (`Day05_SQL_Lab`). Type the file name `Day05_Joined_Sales.csv` → **Export**.
   All 500 rows are saved, even though the result panel shows only 50 rows per page.

Import into Excel

1. Open a **new blank workbook** in Excel.
2. **Data** → **From Text/CSV** (on the left of the Data tab).
3. Choose `Day05_Joined_Sales.csv` → **Import** (on some PCs the button says **Open**).
4. A preview window appears. Check that the columns are split correctly (**Delimiter: Comma**) → click **Load**.
5. Excel creates a new sheet with a formatted **Table** (blue banded rows) of 500 rows.

Check: do Excel and SQL agree? *(Integrated: Apply quality standards.)* In empty cells beside the table:

Check	Excel formula	Must equal (SQL)
Number of rows	`=ROWS(Day05_Joined_Sales[sale_id])`	500 (Q1)
Total revenue	`=SUM(Day05_Joined_Sales[total_amount])`	49,319 (Q2)
GCash revenue	`=SUMIF(Day05_Joined_Sales[payment_method],"GCash",Day05_Joined_Sales[total_amount])`	14,253 (Q4)

If a number does not match, **don't continue**. Check that you exported the right query, and that Excel split the columns correctly.

> **Why not open the CSV by double-clicking?** Double-clicking lets Excel *guess* the data types. It may turn codes like `0917…` into numbers and drop the leading zero, or change dates. *Data → From Text/CSV* shows a preview first, so you stay in control.

Part 6 — Pair Programming *(Integrated: Lead small teams)*

> **Sa simpleng salita:** Two people, **one computer**. The **driver** types. The **navigator** reads the question, watches for mistakes, and thinks one step ahead. Switch roles every 2 queries.

Do	Don't
Navigator reads the question aloud first	Navigator grabs the mouse
Driver says what they are typing ("FROM sales AS s...")	Driver types silently
Switch roles every 2 queries; record it in the **pair log**	One person does everything
Disagree politely: "Let's run it and look at the row count"	Argue without testing

---

Quick Reference

Need	Write
Count rows	`SELECT COUNT(*) AS n FROM sales;`
Count different values	`COUNT(DISTINCT customer_id)`
Total / average / smallest / biggest	`SUM(x)` · `ROUND(AVG(x), 2)` · `MIN(x)` · `MAX(x)`
Column alias / table alias	`SUM(x) AS total` · `FROM sales AS s`
Combine matching rows	`FROM sales AS s INNER JOIN products AS p ON s.product_id = p.product_id`
Keep all of the left table	`FROM products AS p LEFT JOIN sales AS s ON p.product_id = s.product_id`

Need	Write
Find "no match"	`… LEFT JOIN … WHERE s.sale_id IS NULL`
Export	Result panel → **EXPORT** → **Save results as CSV**
Import into Excel	**Data** → **From Text/CSV** → Import (or Open) → **Load**

Glossary

Term	Formal definition	Sa simpleng salita
Aggregate function	A function that returns one value computed from many rows	Pinagsama-samang sagot mula sa maraming row
Alias	A temporary name for a column or table in a query	Palayaw ng column o table
JOIN	Combining rows of two tables using a related column	Pagdugtong ng dalawang table
INNER JOIN	Returns only the rows that have a match in both tables	Yung may kapares lang
LEFT JOIN	Returns all rows of the left table, with NULLs where the right table has no match	Lahat ng nasa kaliwa, kahit walang kapares
Join condition (ON)	The rule that says which rows match, usually FK = PK	Ang batayan ng pagtutugma
Flat / joined dataset	One table that already contains the related names and categories	Isang buong table, handa na para sa Excel
CSV	A text file of rows with comma-separated values	Simpleng text na table
Export / import	Saving data out of one program / bringing it into another	Ilabas / ipasok ang data
Reconciliation	Checking that totals from two tools or sources agree	Pagtutugma ng total
Pair programming	Two people working together on one computer as driver and navigator	Dalawahang pag-code

Key Takeaways

- Aggregates turn many rows into **one answer**: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. `WHERE` runs first.
- Aliases make queries **shorter** and headings **clearer**.
- `INNER JOIN` = matched rows only. `LEFT JOIN` + `IS NULL` = find what is **missing**.
- Export with **CSV**, import with **Data** → **From Text/CSV**, then **check that the totals match**.
- Two heads are better than one. Drive, navigate, **switch**.

Next session (Day 6): Clean Data Sets · duplicates, missing values, inconsistent formats, and Power Query.